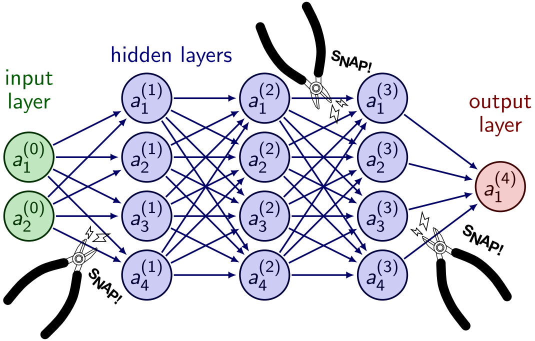
Structure and Interpretation of Deep Neural Networks

Pavanakumar Mohanamuraly



हिरण्मयेन पात्रेण सत्यस्यापिहितं मुखम् ।

तत्त्वं पूषन्नपावृणु सत्यधर्माय दृष्टये ॥१५॥

The door of the Truth is covered with a golden lid. Remove, O Sun (Nourisher), the covering, that I, the worshipper of the Truth may behold it.

ॐ शान्तिः शान्तिः शान्तिः ॥

Deep Learning

- Machine learning deals with algorithms that can learn from data
- Deep learning is a specific kind of machine learning
- Consider the following
- Task T : Classification, Regression, Denoising, etc.
- Performance measure P : Measure of accuracy or performance of of task T using the model
- Experience E : Dataset or test set used for training

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. [1]

- Supervised learning : Labelled data
- Unsupervised learning : Raw data (denoising, clustering)

[1] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill, New York

Deep Neural Network

$$\mathbf{x}_{out} = g(\mathbf{x}_{in}) = \sigma(\mathbf{b} - \mathbf{A}\mathbf{x}_{in})$$

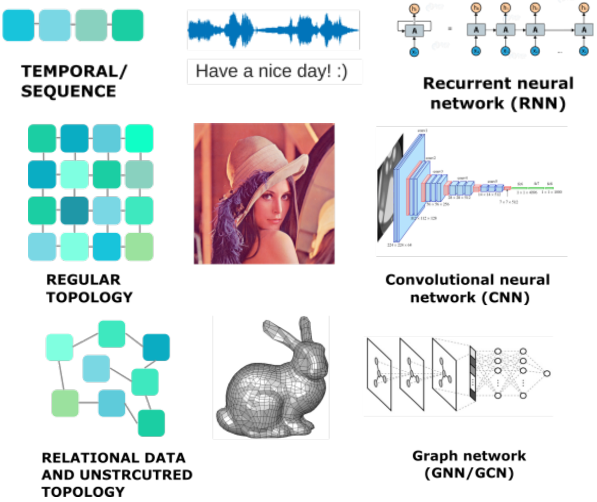
Efficient gradients from *back-propagation* (AD)

$$\mathbf{A} \leftarrow \mathbf{A} + \left(\frac{\partial g}{\partial \mathbf{A}}\right)^T \mathbf{x}_{out}$$

$$\mathbf{b} \leftarrow \mathbf{b} + \left(\frac{\partial g}{\partial \mathbf{b}}\right)^T \mathbf{x}_{out}$$

Deep combinatorics produces rich representations

Parameters optimised using **gradient-based** methods



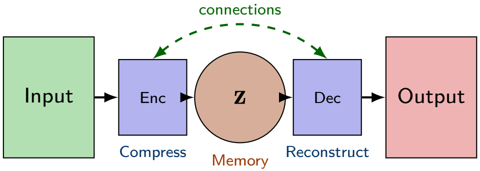
Encoders: Memory Units of Deep Networks

Key Insight: Deep networks can be viewed as:

- **Encoder units** – store compressed representations (memory)
- **Connecting units** – transform and evolve these representations toward solutions

The *efficacy* of a network depends on:

1. Quality of encoded memories
2. Structure of connections between encoders



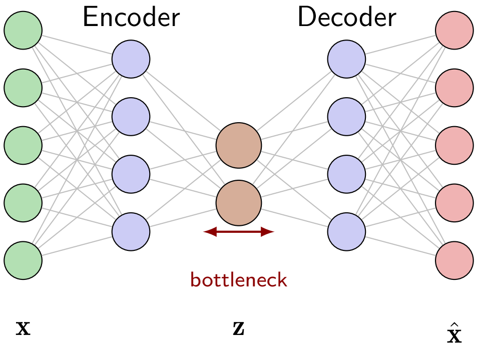
Memory $\mathbf{z}$ captures essential features; *connections* govern information flow

Autoencoders: Learning Efficient Representations

$$\begin{aligned} \text{Encoder:} \quad & \mathbf{z} = f_\theta(\mathbf{x}) \\ \text{Decoder:} \quad & \hat{\mathbf{x}} = g_\phi(\mathbf{z}) \\ \text{Loss:} \quad & \mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 \end{aligned}$$

Key Properties:

- *Bottleneck* forces compression $\dim(\mathbf{z}) \ll \dim(\mathbf{x})$
- Learns *latent features* without supervision
- Variants: VAE, Sparse AE, Denoising AE

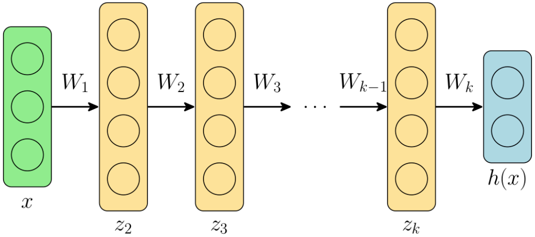


Applications: Dimensionality reduction, anomaly detection, generative models, feature learning for downstream tasks

Deep Feed-forward Network

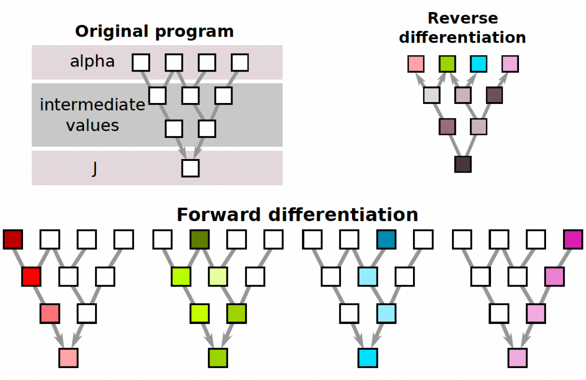
Typical k-layer deep FF network

$$\begin{aligned} \mathbf{z}_1 &= \mathbf{x} \\ \mathbf{z}_{i+1} &= \sigma(\mathbf{W}_i \mathbf{z}_i + \mathbf{b}_i), \quad {\scriptstyle i=1,\ldots,k-1} \\ \mathbf{h}(\mathbf{x}) &= \mathbf{W}_k \mathbf{z}_k + \mathbf{b}_k \end{aligned}$$



- $\mathbf{x}$ are the input features
- $\mathbf{z}_i$ are called hidden layers (intermediate outputs)
- $\mathbf{h}$ are the output features
- k is the depth of the feedforward network

Problem of Gradient Checkpointing



Reverse AD needs to restore memory states in the reverse order of their normal appearance

Problem of Gradient Checkpointing

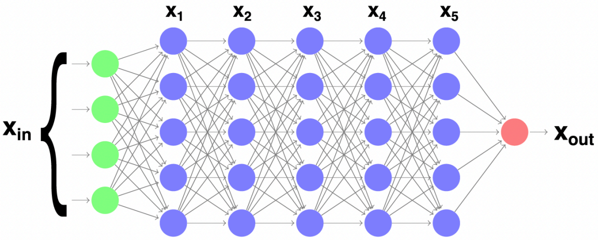
$$g(\mathbf{x}_{in}) := \mathbf{x}_{out} = \sigma(\mathbf{A}\mathbf{x}_{in} + \mathbf{b})$$

$$\bar{\mathbf{A}} \leftarrow \bar{\mathbf{A}} + \left(\frac{\partial g}{\partial \mathbf{A}}\right)^T \bar{\mathbf{x}}_{out}$$

$$= \bar{\mathbf{A}} + \left(\frac{\partial R}{\partial \mathbf{A}}\right)^T \left(\frac{\partial \sigma}{\partial R}\right)^T \bar{\mathbf{x}}_{out}$$

$$\bar{\mathbf{b}} \leftarrow \bar{\mathbf{b}} + \left(\frac{\partial g}{\partial \mathbf{b}}\right)^T \bar{\mathbf{x}}_{out}$$

$$= \bar{\mathbf{b}} + \left(\frac{\partial R}{\partial \mathbf{b}}\right)^T \left(\frac{\partial \sigma}{\partial R}\right)^T \bar{\mathbf{x}}_{out}$$



$$\bar{\mathbf{A}} \xleftarrow[+]{\mathbf{x}_{in}} \bar{\mathbf{A}}_1 \xleftarrow[+]{\mathbf{x}_1} \bar{\mathbf{A}}_2 \xleftarrow[+]{\mathbf{x}_2} \bar{\mathbf{A}}_3 \xleftarrow[+]{\mathbf{x}_3} \bar{\mathbf{A}}_4 \xleftarrow[+]{\mathbf{x}_4} \bar{\mathbf{A}}_5 \xleftarrow[+]{\mathbf{x}_5} \frac{\mathbf{x}_{out}}{\bar{\mathbf{x}}_{out}} \quad \bar{\mathbf{A}} = 0$$

$$\bar{\mathbf{b}} \xleftarrow[+]{\mathbf{x}_{in}} \bar{\mathbf{b}}_1 \xleftarrow[+]{\mathbf{x}_1} \bar{\mathbf{b}}_2 \xleftarrow[+]{\mathbf{x}_2} \bar{\mathbf{b}}_3 \xleftarrow[+]{\mathbf{x}_3} \bar{\mathbf{b}}_4 \xleftarrow[+]{\mathbf{x}_4} \bar{\mathbf{b}}_5 \xleftarrow[+]{\mathbf{x}_5} \frac{\mathbf{x}_{out}}{\bar{\mathbf{x}}_{out}} \quad \bar{\mathbf{b}} = 0$$

- Unsteady CFD adjoints-gradient
- Deep neural networks

Checkpointing Strategy : The Two Extremes

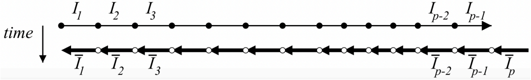
Store All:

- Memory $\propto T$ (tape length)
- Time: $1\times$ forward $+ 1\times$ reverse
- Prohibitive for large T!

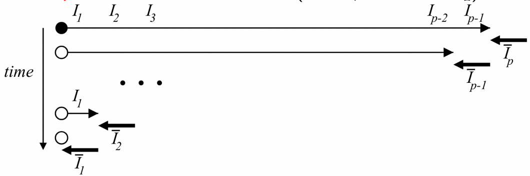
Recompute All:

- Memory $\propto R$ (state size)
- Time: $O(n^2)$ – restart from s_0 each time
- Prohibitive runtime!

Store-all and undo progressively (CPU ↘ ; Mem ↗) :



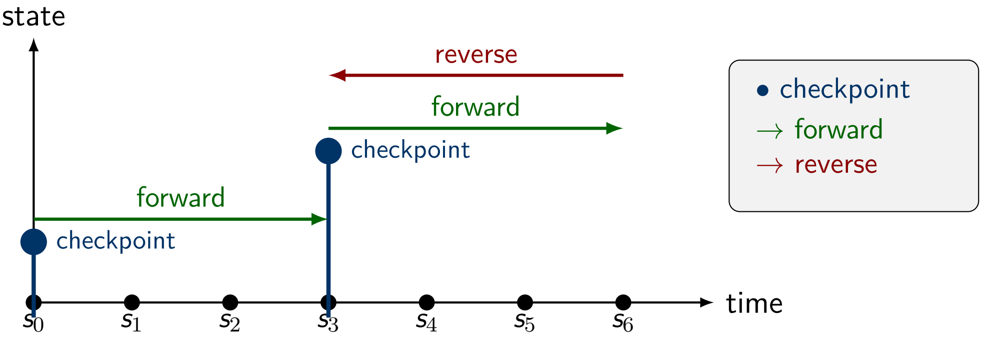
Recompute-all from initial state (CPU ↗ ; Mem ↘) :



Can we do better? Yes! – The REVOLVE Algorithm (Griewank, 1992)

REVOLVE: Optimal Checkpointing Strategy

Key Idea: Use d snapshots (checkpoints) and allow t extra forward sweeps



- Save state at strategic checkpoints, recompute segments as needed
- Trade-off: More checkpoints ⇒ less recomputation (but more memory)
- **Question:** What is the *optimal* checkpoint placement?

Griewank's Key Result : The Binomial Solution

With d snapshots and t extra forward sweeps, the *maximum* number of steps η that can be reversed is:

$$\boxed{\eta(d,t) = \binom{d+t}{t} = \frac{(d+t)!}{d\,!\,t\,!}}$$

Examples:

- $d = 3, t = 5 \Rightarrow \eta = \binom{8}{5} = 56$ steps
- $d = t = 12 \Rightarrow \eta \approx 10^6$ steps!

Complexity grows logarithmically:

$$d = t \approx \log_4(\eta)$$

Pascal's Triangle Structure

$$\begin{array}{r ccccccccccc}
d+t=0 & & & & & & 1 & & & & & \\
1 & & & & & 1 & & 1 & & & & \\
2 & & & & 1 & & 2 & & 1 & & & \\
3 & & & 1 & & 3 & & 3 & & 1 & & \\
4 & & 1 & & 4 & & \color{darkred}{6} & & 4 & & 1 & \\
5 & 1 & & 5 & & 10 & & 10 & & 5 & & 1
\end{array}$$

$d = t = 2$

$$\eta(d,t) = \eta(d-1,t) + \eta(d,t-1)$$

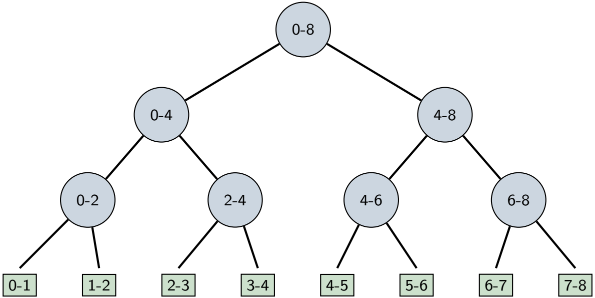
(Each entry = sum of two above)

Treeverse Recursive Algorithm

1. Take snapshot
2. Advance to midpoint
3. Reverse second half
4. Restore snapshot
5. Reverse first half

Optimal partition:

$$\kappa = \left\lceil \frac{\delta \cdot \sigma + \tau \cdot \phi}{\tau + \delta} \right\rceil$$

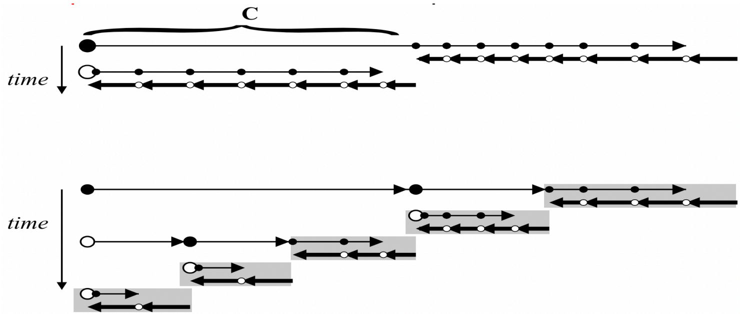


Recursive call tree

where, δ=remaining snapshots, τ=remaining sweeps

Result: Memory $\sim O(d \cdot R)$, Time $\sim O((1 + t) \cdot W)$ where $d + t \sim \log(\eta)$

Binary Checkpointing Summary

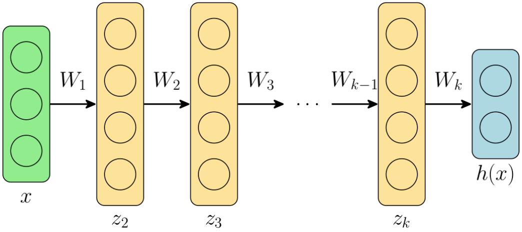


- Use d snapshots and allow t extra forward sweeps
- **REVOLVE** (Griewank): optimal binomial partitioning $\eta(d,t) = \binom{d+t}{t}$
- Memory and CPU overhead grow like $\mathbf{O}(\log(\eta))$ – *logarithmic!*
- Enables adjoint computation for virtually **any problem size**

Connection with implicit functions*

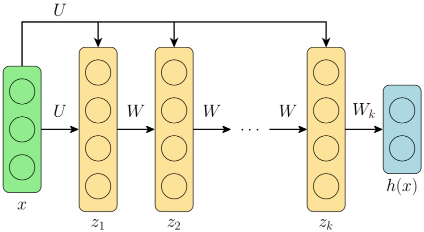
Typical k-layer deep network

$$
\begin{aligned}
\mathbf{z}_1 &= \mathbf{x} \\
\mathbf{z}_{i+1} &= \sigma(\mathbf{W}_i \mathbf{z}_i + \mathbf{b}_i), \quad {\scriptstyle i=1,\ldots,k-1} \\
\mathbf{h}(\mathbf{x}) &= \mathbf{W}_k \mathbf{z}_k + \mathbf{b}_k
\end{aligned}
$$



Consider same weight/bias

$$
\begin{aligned}
\mathbf{z}_1 &= \mathbf{0} \\
\mathbf{z}_{i+1} &= \sigma(\mathbf{W} \mathbf{z}_i + \mathbf{U}\mathbf{x} + \mathbf{b}) \\
\mathbf{h}(\mathbf{x}) &= \mathbf{W}_k \mathbf{z}_k + \mathbf{b}_k
\end{aligned}
$$



For ∞ layers with same weights we get a non-linear fixed-point

$$\mathbf{z}^* = \sigma(\mathbf{W}\mathbf{z}^* + \mathbf{U}\mathbf{x} + \mathbf{b}) \quad \text{or} \quad \mathbf{z} = f(\mathbf{z}, \mathbf{a})$$

* *Deep Equilibrium Models*, Bai et. al., NeurIPS 2019.

Implicit Functions and Attractive Fixed-points

- Christianson[2] in his seminal work shows that a fixed-point of the form

$$z := \Phi(z, x) \tag{1}$$

- Φ is well behaved iterative construction that converges to the value z_*, the adjoint-gradient only requires values from the last iteration

$$y := f(z_*, x) \tag{2}$$

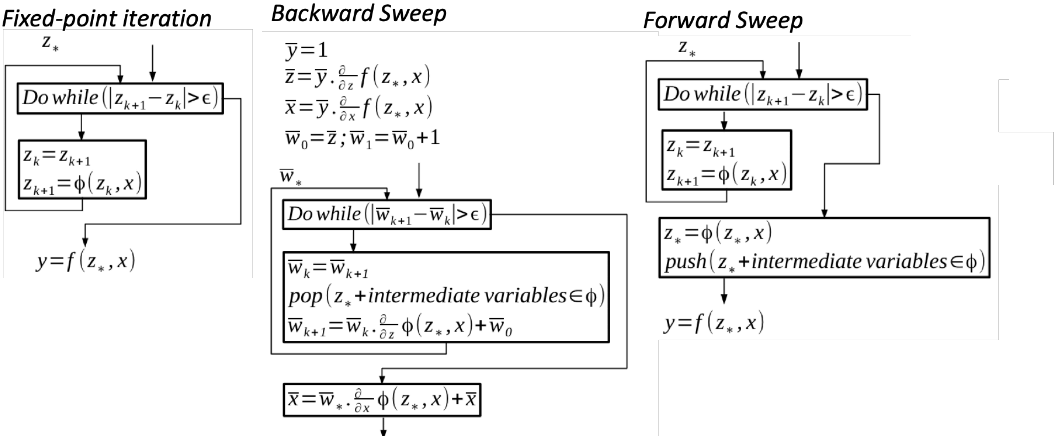
- In BWD sweep, adjoint loop solves the adjoint FP equation

$$\bar{w} := \bar{w} \cdot \frac{\partial}{\partial z}\Phi(z_*, x) + \bar{z} \tag{3}$$

- $\bar{w}$ is returned by the adjoint of downstream computation of f

[2] *Reverse accumulation and attractive fixed points*, OMS, vol. 3(4), 1992.

Implicit Functions and Attractive Fixed-points[3]



[3] *Implementation and measurements of an efficient Fixed Point Adjoint*, A Taftaf, et. al., EUROGEN 2015

Deep combinatorics?

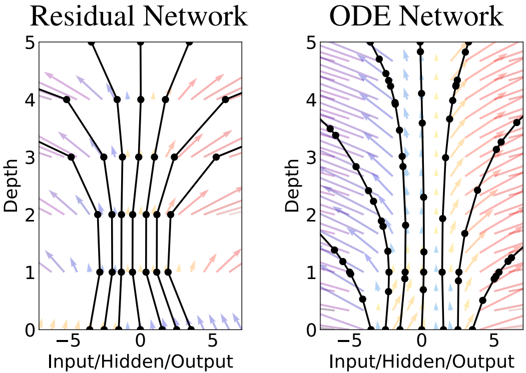
Strong connect between ODEs and Neural Networks[4]

ResNet/RNN are sequence of transformations on hidden state $\mathbf{h}$,

$$\mathbf{h_{t+1}} = \mathbf{h_t} + f(\mathbf{h_t}, \theta_t)$$

In the limit of ∞ layers

$$\frac{d\mathbf{h}}{dt} = f(\mathbf{h}, t, \theta)$$



[4] *Neural Ordinary Differential Equations*, Chen et. al., NeurIPS 2018.

Deep Network Training (Vanishing Gradients)

Chain Rule in Deep Networks: For n-layer network, gradient at layer 1:

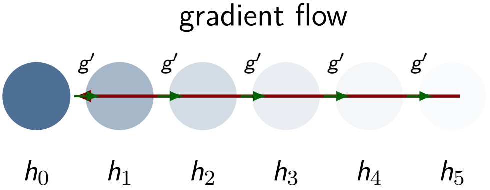
$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial h_n} \cdot \underbrace{g'_{n-1} \cdot g'_{n-2} \cdot \ldots \cdot g'_1}_{n-1 \text{ terms}} \cdot x_0$$

Analytical Example:

Let $g'_i = \sigma'(\cdot)$ for sigmoid/tanh where $|g'_i| \leq 0.25$

Layers	Gradient Scale
$n = 5$	$(0.25)^4 \approx 0.004$
$n = 10$	$(0.25)^9 \approx 10^{-6}$
$n = 20$	$(0.25)^{19} \approx 10^{-12}$
$n = 50$	$(0.25)^{49} \approx 10^{-30}$

Gradients vanish exponentially!



Circle opacity $\propto$ gradient magnitude

Early layers receive near-zero gradients

Exploding Gradients: The Other Extreme

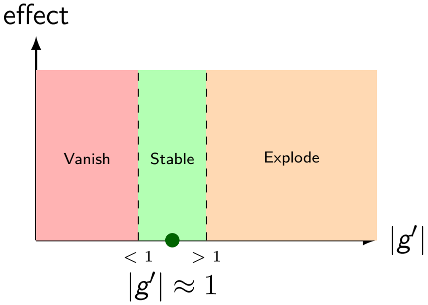
When $|g_i'| > 1$: Gradients grow exponentially! **Analytical Example:**

Let $g_i' = 2$ (e.g., poorly initialized ReLU)

Layers	Gradient Scale
$n = 5$	$2^4 = 16$
$n = 10$	$2^9 = 512$
$n = 20$	$2^{19} \approx 5 \times 10^5$
$n = 50$	$2^{49} \approx 5 \times 10^{14}$

Causes NaN, training divergence!

The Goldilocks Zone:



Goal: Keep $\prod_i |g_i'| \approx 1$

Solution: Careful weight initialization (Xavier/He) + architecture design (ResNets)

Xavier[5] Initialisation - variance constant across layers

For layer l: $\quad y^{(l)} = W^{(l)} x^{(l-1)}$

Variance propagation:

$$\mathrm{Var}(y^{(l)}) = n^{(l-1)} \cdot \mathrm{Var}(W^{(l)}) \cdot \mathrm{Var}(x^{(l-1)})$$

For $\mathrm{Var}(y^{(l)}) = \mathrm{Var}(x^{(l-1)})$, we need:

$$\boxed{\mathrm{Var}(W^{(l)}) = \frac{1}{n^{(l-1)}}}$$

Xavier (Glorot) Init:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

He Init (for ReLU):

ReLU zeros out half the values, so:

$$\boxed{\mathrm{Var}(W^{(l)}) = \frac{2}{n^{(l-1)}}}$$

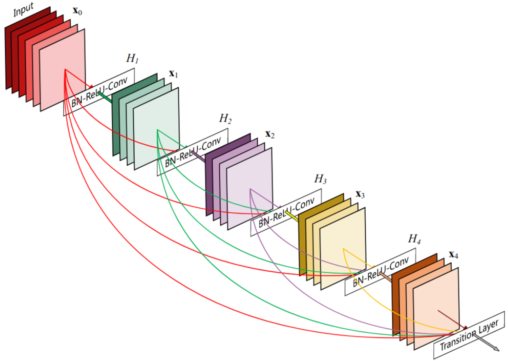
Practical Rules:

- **tanh/sigmoid:** Xavier
- **ReLU/Leaky ReLU:** He
- **SELU:** LeCun init

Result: Gradients stay $\approx O(1)$ across all layers!

[5] *Understanding the difficulty of training deep feedforward neural networks*, PMLR 9:249-256, 2010.

Skip Connections to the Rescue[6] [7]



- Skip connections one of the important innovations in DL
- Provide direct gradient paths, allowing gradients to flow more easily to earlier layers
- Makes optimization landscape smoother by allowing gradients to flow more easily during training
- DenseNet, ResNet, UNets, Transfomers employ them

[6] *The Shattered Gradients Problem: If ResNets are the Answer, Then What is the Question?*, Veit et al., NIPS 2016.

[7] *Attention Is All You Need*, A Vaswani, et. al., 2017

What is an Optimizer?

Problem: How do we update weights to reduce error?

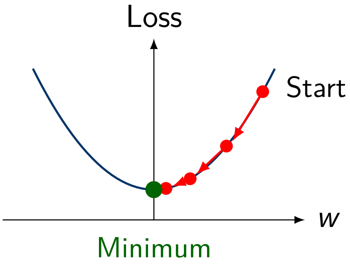
Solution: Use gradient descent!

- Compute gradient (slope) of loss
- Move weights in opposite direction
- Repeat until loss is small

Update Rule:

$$w_{new} = w_{old} - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

where η is the **learning rate**



Each step moves toward lower loss

SGD: Stochastic Gradient Descent

The Simplest Optimizer

Update Rule:

$$w = w - \eta \cdot \frac{\partial \mathsf{Loss}}{\partial w}$$

Pros: Simple, low memory

Cons: Slow, sensitive to η

“Stochastic” $=$ uses mini-batches

Adam: Adaptive Moment Estimation

A Smarter Optimizer

Adapts learning rate per weight:

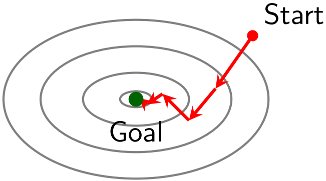
- Tracks **momentum**
- Tracks **velocity**
- Auto-adjusts step size

Pros: Fast, works out-of-box

Cons: More memory

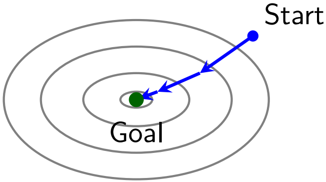
Rule: Start with Adam!

SGD vs Adam: Visual Comparison



SGD Path

Zigzag path, many steps



Adam Path

Smoother path, fewer steps

Adam uses momentum to avoid zigzag behavior and converges faster in most cases

Learning Rate (η): The Most Important Hyperparameter

Too Large:

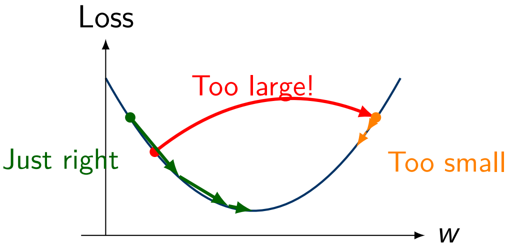
- Overshoots minimum
- Training diverges (loss $\to \infty$)

Too Small:

- Very slow training
- May get stuck

Just Right:

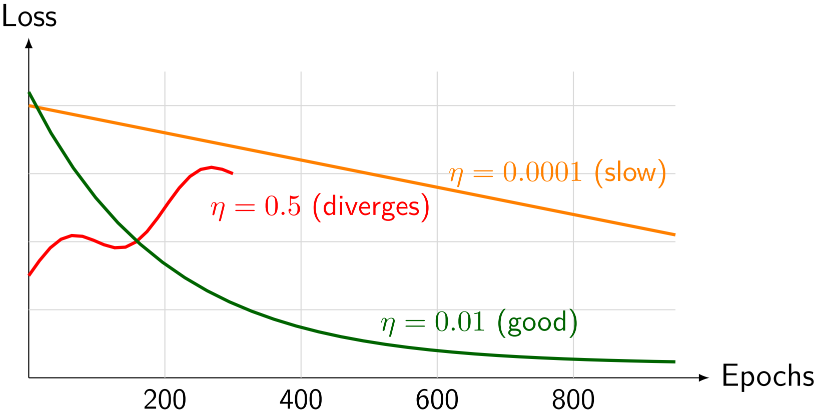
- Smooth convergence
- Reaches good solution



Typical values: SGD: 0.01–0.1   Adam: 0.001–0.0001

Effect of Learning Rate on Training

Training Loss vs Epochs for Different Learning Rates



Good Learning Rate (η): Shows smooth, steady decrease in loss

ॐ (३) क्रतो स्मर कृतं स्मर क्रतो स्मर कृतं स्मर ॥१७॥

O my mind, remember all that has been done. Remember – remember all that has been done.

ॐ शान्तिः शान्तिः शान्तिः ॥